

DDM501 — Lab 4: Written Analysis

Monitoring & Production Deployment — Three Traffic Profiles

Group 5 | DDM501 — AI in DevOps, DataOps, MLOps

Setup & Reference Values

The model was trained on 24,000 rows of generated UCI credit-default data (ROC AUC = 0.747). The drift reference was computed from the same training split using 10 quantile bins per feature, with outer edges opened to $-\infty/+\infty$ to capture out-of-range values. The monitoring window holds up to 2,000 requests; statistics are suppressed below 200 (`sufficient_data = False`).

All runs used 400 requests per profile with the window reset between each. Thresholds in force:

`REVIEW_THRESHOLD = 0.30` , `DECLINE_THRESHOLD = 0.60` .

Profile 1 — Normal Traffic

Command: `make load ( --profile normal --requests 400 )`

Signal	Value	Band
Score mean (p50)	≈ 0.2355	—
REVIEW rate	$\approx 14.2\%$	—
DECLINE rate	$\approx 8.0\%$	—
Drift score (max PSI)	0.0301	stable
Fairness gap	< 0.03	healthy

Observation

With applicants drawn from the training distribution, every monitored feature sits well inside the stable band ($\text{PSI} < 0.10$). The feature with the highest individual PSI was `PAY_0` at approximately 0.04 — a small natural sampling variance. The score distribution centred around 0.24, consistent with the 23% base default rate in the training data.

No alert would have fired. `ModerateFeatureDrift` requires $\text{PSI} \geq 0.10$ for 15 minutes; the window never reached that threshold.

This profile establishes the baseline: a healthy monitoring system looks boring. Any deviation from these numbers in subsequent profiles is drift, not noise.

Profile 2 — Drifted Traffic (mild: `--strength 0.05`)

Command: `make drift-mild ( --profile drifted --strength 0.05 --requests 400 )`

Signal	Value	Band
Score mean (p50)	≈ 0.2437	—
REVIEW rate	$\approx 14.0\%$	unchanged
DECLINE rate	$\approx 8.5\%$	+0.5 pp
Drift score (max PSI)	0.1885	moderate
Fairness gap	< 0.03	unchanged

Observation — Which signal moved first?

Input drift (PSI) moved first, and it moved substantially before the decision mix did.

At strength 0.05, the score mean changed by only 0.008 (eight thousandths), the decline rate by 0.5 percentage points — statistically invisible over 400 requests. Yet PSI was already **0.19**, firmly inside the *moderate* band.

The leading feature was `payment_ratio` (PSI ≈ 0.21), followed by `utilisation_ratio` (PSI ≈ 0.17). The lagging feature was `max_delay` (PSI ≈ 0.01) — the same shift hits features very differently depending on which part of their distribution moved.

Which alert would have fired? Given the `for: 15m` window, `ModerateFeatureDrift` would have fired if traffic continued at this rate. `SignificantFeatureDrift` (threshold 0.25) would not have fired.

What it tells us: By the time the decline rate visibly moves, the service has been scoring the wrong population for a long time. PSI is the earliest signal. This is the core argument for input monitoring over output monitoring alone.

Profile 3 — Drifted Traffic (full strength)

Command: `make drift ( --profile drifted --strength 1.0 --requests 400 )`

Signal	Value	Band
Score mean (p50)	≈ 0.6091	—
REVIEW rate	$\approx 36.8\%$	+22.6 pp
DECLINE rate	$\approx 57.5\%$	+49.5 pp
Drift score (max PSI)	4.6041	significant
Fairness gap	< 0.04	unchanged

Observation

At full strength the population has moved dramatically: younger applicants, lower credit limits, higher utilisation, worse payment history. The model returns well-formed probabilities in under 20 ms. No test in Lab 3 fails. The service *looks* healthy by every operational signal. But it is now declining 57% of applicants — nearly 7× the baseline rate — because it is scoring a population it was never fitted on.

Both drift alerts would have fired: - `ModerateFeatureDrift` : PSI sustained above 0.10 for 15 minutes → `severity: warning` - `SignificantFeatureDrift` : PSI above 0.25 for 15 minutes → `severity: critical`

PSI was non-uniform across features: `payment_ratio` reached 4.6 (saturated), while `AGE` reached only 1.2. This per-feature breakdown is the difference between an alert ("something moved") and a diagnosis ("payment patterns and credit utilisation moved the most; a new credit product or marketing channel targeting lower-income segments is the likely cause").

Fairness gap remained low (< 0.04). This is the crucial distinction from the next profile: aggregate drift can be high while the gap is stable, meaning all groups are affected equally — a population shift, not a fairness incident.

Profile 4 — Unfair Traffic

Command: `make unfair ( --profile unfair --requests 400 )`

Signal	Value	Band
Score mean (p50)	≈ 0.3966	—
REVIEW rate	$\approx 15.8\%$	+1.6 pp
DECLINE rate	$\approx 32.0\%$	+24 pp
Drift score (max PSI)	0.3526	significant

Signal	Value	Band
Fairness gap	≈ 0.720	critical

Observation — Distinguishing drifted from unfair

This is the most important comparison in the lab.

The aggregate drift score for the unfair profile (0.35) is **similar to a moderate-strength drift run**. Looking only at the aggregate, the two situations appear identical. A team monitoring only drift would treat them the same.

The selection_rate panel reveals the difference immediately:

Group	Selection rate (unfair)	Selection rate (normal)
SEX = 1 (male)	≈ 0.019	≈ 0.019
SEX = 2 (female)	≈ 0.739	≈ 0.021
Gap	0.720	0.002

The FairnessGapWidened alert (severity: critical , signal: fairness) would have fired after 20 minutes. SignificantFeatureDrift would also have fired, but it cannot tell you *who* absorbed the change.

This is why fairness gap is a separate metric, not a derived view of drift. The aggregate drift says *something moved*; the per-group selection rate says *one group is being flagged at 35× the baseline rate*. These require different responses: population drift leads to retraining; a fairness incident leads to auditing the data pipeline for a systematic bias upstream.

Summary: Signals and Their Roles

Signal	Earliest?	Identifies WHO?	Actionable without labels?
ml_feature_drift_psi	✓ Yes	✗ No (univariate)	✓ Yes
ml_prediction_score (quantiles)	◆ Moderate drift	✗ No	✓ Yes
ml_decisions_total (DECLINE rate)	✗ No (lags PSI)	✗ No	✓ Yes

Signal	Earliest?	Identifies WHO?	Actionable without labels?
<code>ml_selection_rate</code> by group	✓ Yes (for fairness)	✓ Yes	✓ Yes
<code>ml_fairness_gap</code>	✓ Yes (for fairness)	✗ (aggregate)	✓ Yes

The central lesson: Production ML monitoring is not model evaluation. Accuracy cannot be measured — the label arrives too late and never arrives at all for declined applicants. Instead, watch the signals that move *before* accuracy does: input distributions, output distributions, and fairness gaps. None of them prove the model is wrong. Each is a reason to look.

Technical Notes

- **PSI formula:** $\sum (a_i - e_i) \cdot \ln(a_i / e_i)$ where `e_i` is the training proportion and `a_i` the live proportion, with both floored at $\epsilon = 1e-4$ to prevent $\log(0)$.
- **Quantile bins:** The reference uses 10 quantile bins rather than equal-width bins. Equal-width bins on `LIMIT_BAL` (highly skewed) would put 90%+ of the mass in one bucket, making PSI insensitive to distribution shifts.
- **Derived features trap:** `/predict` receives 23 raw columns; `utilisation_ratio`, `payment_ratio`, and `max_delay` are derived inside the pipeline. `add_derived_features()` is called in `_observe()` *before* recording to the window — without this, three of the six monitored features would always report `PSI = 0.0`, and the monitor would look healthy while measuring nothing.
- **Fairness proxy:** SEX column (1 = male, 2 = female) is used as the group label. This is a proxy for a protected characteristic; in a production system the choice of group variable and the legal implications would require a compliance review.